

Guía de uso

Cómo arrancar el sistema de Lisart, cómo correrlo, cómo editar los datos y cómo cambiarlo cuando haga falta.

Actualizada el 10 de septiembre de 2026

Este documento se genera desde `docs/guia_de_uso.md`. No se edita a mano: se regenera con `lisart guia guia`.

Esta guía es **operativa**: pasos en orden, para hacerlos. El *por qué* de cada decisión está en `convenciones_datos.md`; el detalle de cada tabla, en `modelo_de_datos.md`, que se genera solo; y qué es todo esto y por qué se construyó así, en `documentacion_general.md`; y por dónde entrarle al código, en `mapa_del_codigo.md`.

Contenido

Cómo leer esta guía

0. El mapa en treinta segundos

1. Arrancar (una sola vez)

2. Correrlo: qué se corre y cuándo

3. Editar los datos

4. Cambiar el sistema

5. Cuando algo truena

6. Dónde está cada cosa

7. Palabras que se usan aquí

8. Privacidad

Cómo leer esta guía

No hace falta leerla completa. Cada sección tiene un destinatario distinto:

SECCIÓN	PARA QUIÉN	¿HACE FALTA SABER PROGRAMAR?
1. Arrancar	Quien instala el sistema en una computadora nueva	Un poco: hay que escribir comandos
2. Correrlo	**Quien lo usa cada semana**	No. Son comandos que se copian y se pegan
3. Editar los datos	**Quien corrige o captura información**	No. Casi todo se hace en pantalla
4. Cambiar el sistema	Quien toca el código	Sí
5. Cuando algo truena	Cualquiera. Es la tabla de "qué significa este error"	No
6. Dónde está cada cosa	Quien busca un archivo	No
7. Palabras que se usan aquí	Cualquiera, cuando algo suene raro	No
8. Privacidad	**Cualquiera que comparta un archivo del sistema**	No

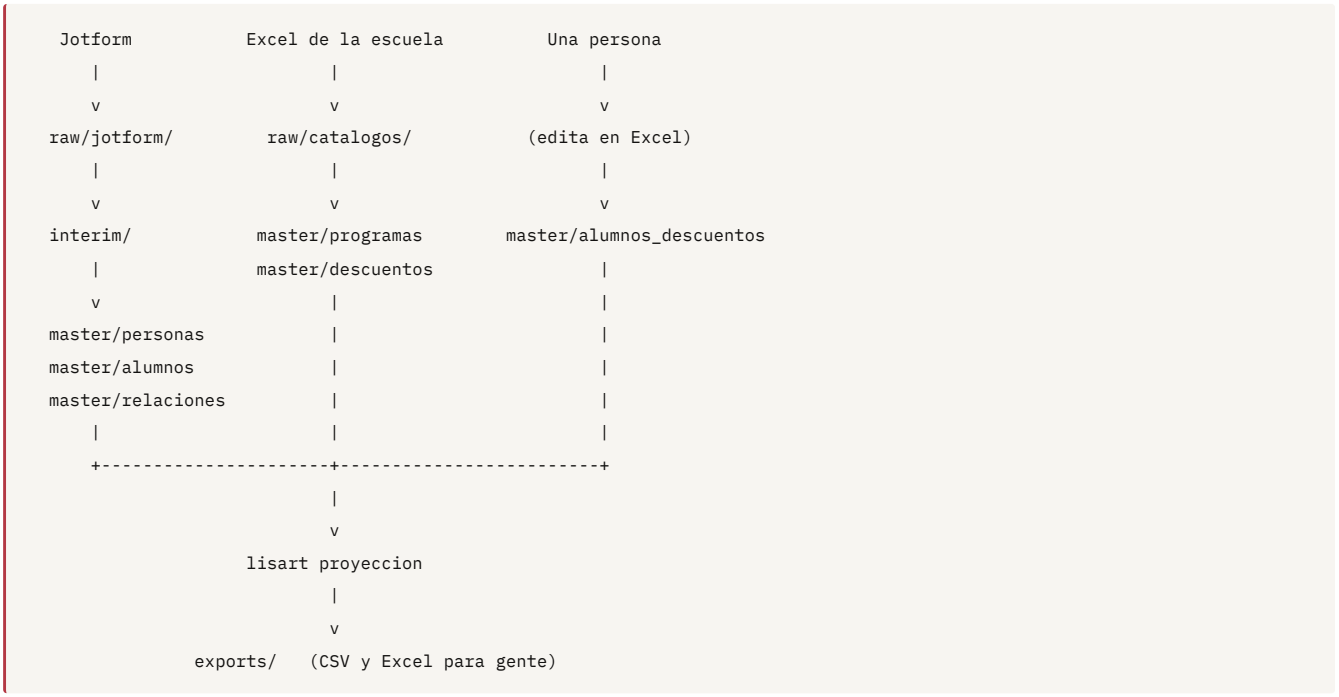
Si la escuela solo va a **usar** el sistema, con las secciones 2, 3 y 5 basta.

Si algo suena a chino, está explicado en la sección 7. Palabras como *terminal*, *comando*, *CSV* o *capa* tienen ahí su traducción.

Tres cosas que valen para todo lo demás

1. **Los comandos se escriben en la terminal**, esa ventana donde se teclea en vez de dar clic. Se abre desde el menú de la computadora buscando "Terminal" (o "Ubuntu", en Windows con WSL).
2. **Correr un comando de más casi nunca hace daño.** Este sistema está hecho para preferir no hacer nada antes que hacer algo dudoso: si algo no le cuadra, se detiene y lo dice, en vez de escribir a medias.
3. **Nada se sobrescribe sin dejar copia.** Si se guardó algo por error, se puede volver atrás (sección 3.4).

0. El mapa en treinta segundos



Tres ideas que explican todo lo demás:

1. `master/` **es la verdad.** Es lo único que se respalda.
2. **Nada llega a** `master/` **sin pasar por la capa** `lisart`, que le pone tipos, llaves y validación a unos archivos que de fábrica no traen nada de eso.
3. `exports/` **es desechable.** Se regenera cuando quieras. Nadie debe editarlo esperando que se conserve.

1. Arrancar (una sola vez)

```
cd /ruta/al/repo

uv venv --python 3.12
uv pip install -e ".[dev,ui]"  # el paquete, las pruebas y la pantalla

cp .env.example .env           # y llenar la llave de Jotform
```

Sin `uv`: `python -m venv .venv` y `pip install -e ".[dev]"` hacen lo mismo.

Los comandos de aquí en adelante se escriben `.venv/bin/lisart ...`. Si prefieres escribir solo `lisart`, activa el entorno primero:

```
source .venv/bin/activate      # en Windows: .venv\Scripts\activate
```

Comprobar que quedó bien

```
lisart init          # crea data/ con las carpetas y los CSV vacíos que falten
lisart validate      # debería decir "Todo en orden"
lisart describe      # lista las 17 tablas
```

`lisart init` **no pisa nada**: solo crea lo que falta. Se puede correr las veces que sea.

2. Correrlo: qué se corre y cuándo

No hay un "arrancar el sistema". Hay comandos que se corren cuando pasa algo.

CUANDO PASA ESTO	SE CORRE
Entraron inscripciones nuevas (cada semana)	`ingest jotform` + `procesar inscripciones` + `descuentos`
La escuela cambió su Excel de precios	`ingest catalogos`
Cambió quién tiene descuento	`descuentos`
Toca cobrar	`proyeccion --excel`
Hay que ver o corregir un dato	`ui`

2.1 Cada semana — entran inscripciones nuevas

```
lisart ingest jotform           # baja los envíos nuevos
lisart procesar inscripciones   # los convierte en personas, alumnos y relaciones
lisart descuentos               # da de alta a los alumnos nuevos en el archivo
lisart validate
```

Correrlo de más no hace daño: si no hay envíos nuevos, no escribe nada.

Hay que leer lo que imprime. Tres cosas importan de verdad:

- `AVISO: Estos campos no llegaron en ningun envio...` (o un aviso de que un campo llegó con el tipo equivocado). Quiere decir que **alguien movió o insertó una pregunta en Jotform**. El mapeo va por posición, así que los datos pueden estar cayendo en la columna equivocada sin que nada truene. Es lo único que lo detecta: **parar y revisar.**
- `CONFLICTO: ...` — dos envíos dicen cosas distintas de la misma persona. En la primera corrida real los dos casos eran errores de captura de verdad.
- `Aviso: N envio(s) que ya estaban cambiaron en Jotform` — alguien editó o borró un envío. **No se pisan solos.** Para traer esos cambios: `lisart ingest jotform --actualizar`.

Si hace falta rehacer la tabla sin volver a llamar a la API (por ejemplo, después de corregir la especificación de campos):

```
lisart ingest jotform --desde-raw
```

Para agendarlo (Linux/WSL, `crontab -e`, lunes 8am):

```
0 8 * * 1 cd /ruta/al/repo && .venv/bin/lisart ingest jotform && .venv/bin/lisart procesar inscripciones
```

En Windows es el Programador de tareas apuntando al mismo comando.

2.2 Cuando la escuela cambia su Excel de precios

```
lisart ingest catalogos     # lee data/catalogos_lisart.xlsx
lisart validate
```

Guarda una copia con fecha en `data/raw/catalogos/` y reescribe `programas`, `descuentos` y las claves de la escuela en `instrumentos`. La escuela sigue trabajando en su Excel; el sistema lo lee, no lo manda.

2.3 Cuando cambian los descuentos

```
lisart descuentos --revisar     # ver qué hay, sin escribir
lisart descuentos               # ponerlo al día
```

El detalle de cómo se edita ese archivo está en la sección 3.2.

2.4 Cuando toca cobrar

```
lisart proyeccion              # imprime y escribe 3 CSV a data/exports/
lisart proyeccion --excel      # además, el Excel en el formato de la escuela
lisart proyeccion --excel --mes 2026-09
```

Lee tablas y escribe un reporte. **No modifica nada.** Se puede correr todas las veces que se quiera.

Antes de mandarle el archivo a alguien: revisar qué columnas lleva. Hay datos de menores de edad.

2.5 Probar algo sin ensuciar los datos reales

```
lisart demo            # siembra profesores, clases y horarios de prueba
lisart demo --limpiar  # los borra
```

Todo lo sembrado lleva `fuente = "demo"`, columna que está a la vista en cada tabla de la pantalla. Se borra entero con una orden, sin tocar un dato real.

Hace falta porque **no hay ni un profesor ni una clase de verdad todavía**: sin eso no hay forma de ejercitar la agenda ni la reposición.

2.6 Ver y editar en pantalla

```
lisart ui               # la pantalla principal
lisart ui --profesores  # la agenda de profesores (aplicación aparte)
```

La primera pintada tarda. Para cerrarla: `Ctrl+C`, o si se quedó trabada, `fuser -k 8501/tcp` (**no** `pkill -f "streamlit run"`: ese patrón aparece en su propia línea de comando y el comando se mata a sí mismo).

3. Editar los datos

Hay tres caminos, y elegir mal es la forma más fácil de romper algo.

3.1 Lo normal: la pantalla

`lisart ui`, se busca el registro, se edita, se guarda.

Lo que la pantalla garantiza y un editor de texto no:

- una llave foránea se elige de una lista, así que **no se puede escribir una referencia rota**;
- un catálogo cerrado se ofrece como lista desplegable;
- antes de escribir **valida**, y solo si pasa, **respalda** el estado anterior;
- lo que no se debe tocar no se deja tocar: llaves primarias, marcas de tiempo y las tablas de capa `interim`.

Hoy la pantalla **edita**, no crea ni borra registros. Crear necesita generar identificadores y decidir roles; borrar necesita revisar referencias antes.

Los roles: quién es qué

Un rol —alumno, tutor, profesor, empleado, director, proveedor— se asigna en la **ficha de la persona**, en la sección *Roles* del final: se marcan los que tenga y se guarda. Una persona puede tener varios a la vez; el profesor que además es tutor de su hija lleva los dos.

Quitar un rol no borra nada. Se cierra con la fecha de ese día, para que después se pueda saber quién daba clase el semestre pasado. Por eso volver a dárselo lo reactiva en vez de crear un registro nuevo.

La sección **Roles** es la otra mitad: un **directorio**. Se elige un rol y sale la lista completa de quienes lo tienen, con teléfono y correo, ordenable por cualquier columna y lista para copiar a Excel. El interruptor *Incluir a quienes ya no lo tienen* muestra también los cerrados, con la fecha en que dejaron de serlo. Y el botón del final abre la ficha de la persona elegida.

```
¿quiénes son los tutores?   ->  sección Roles
¿qué es esta persona?       ->  sección Personas, en su ficha
```

3.2 A mano, en Excel: solo dos casos

Son las tablas marcadas `hand_edited` en el modelo. Van en UTF-8 **con BOM** para que Excel en español no rompa los acentos, y al leerlas se acepta `VERDADERO` / `FALSO` y `si` / `no`.

a) Los catálogos — `espacios`, `instrumentos`, `conceptos`.

Se abren, se editan, se guardan como CSV UTF-8, y después `lisart validate`. Nadie los llena desde el código a propósito: si un dato de fuera no está en el catálogo, el sistema **avisa** en vez de darlo de alta solo, porque eso llenaría el catálogo de variantes mal escritas sin que nadie lo note.

b) Los descuentos — `data/master/alumnos_descuentos.csv`.

Lista a todos los alumnos inscritos, uno por renglón, con los que tienen descuento arriba. **Solo se editan dos columnas: `descuento` y `notas`.**

PARA	SE HACE
Dar de alta un descuento	escribir su nombre en la columna `descuento`
Quitarlo	borrar esa celda
Un segundo descuento al mismo alumno	duplicar el renglón y cambiarle el descuento

Al terminar, **siempre**:

```
lisart descuentos
```

Eso rellena el nombre del alumno, resuelve la clave del catálogo, siembra el porcentaje, da de alta a los alumnos nuevos y asigna los identificadores que falten. Un renglón nuevo se puede dejar **sin ID y hasta sin** `persona_id`, escribiendo solo el nombre del alumno.

Dos reglas de ese archivo:

- **El porcentaje va en puntos**: `5` es 5%, `100` es beca completa.
- **Manda la celda, no el catálogo.** Se puede pactar un caso particular sin inventar una entrada de catálogo; a cambio, si el descuento familiar sube del 5% al 7%, hay que tocar los renglones que lo tengan. El comando avisa cuando una fila se separó del catálogo.

Si algo no cuadra, el comando **no escribe nada** y dice qué corregir. Es a propósito: se corrige en el mismo archivo y se vuelve a correr.

3.3 Lo que nunca se edita a mano

NO TOCAR	POR QUÉ
`data/raw/**`	Es lo que llegó de afuera, tal cual. Es inmutable. Si un dato llegó mal se corrige en el paso que lo transforma, no aquí.
`data/interim/**`	Se reconstruye entero desde `raw`. Editarlo es trabajo perdido.
`data/exports/**`	Se regenera. Si vas a llenar columnas a mano, trabaja sobre una copia tuya.
`docs/modelo_de_datos.md`	Lo genera `lisart docs`.
Los PDF de `docs/`	Los genera `lisart guia`. Se edita el `.md` de cada uno.
`vault/Tablas/`	Lo genera `lisart vault`. El resto del vault sí es tuyo.

Y desde código o notebook: **nunca `pd.read_csv` ni `df.to_csv` sobre `data/`.** Todo pasa por `read_table` / `write_table` / `upsert` de `lisart.io`, que es donde viven los tipos, la validación y la escritura atómica (o el archivo queda completo, o no se toca).

3.4 Deshacer

Cada vez que se guarda, el estado anterior queda en `data/backups/<tabla>/<tabla>_<fecha>.csv`. Se conservan las últimas 50 copias por tabla.

Para restaurar: en la pantalla, en el desplegable **Respaldos** de esa tabla. Restaurar **también deja copia** de lo que pisa, así que deshacer una restauración también es posible.

Para respaldar a propósito antes de algo grande:

```
lisart backup              # toda la capa master
lisart backup personas     # una sola tabla
```

4. Cambiar el sistema

4.1 Agregar una columna a una tabla

1. Agregar el `Column(...)` en `src/lisart/tables.py`.
2. `make docs` — regenera `docs/modelo_de_datos.md` **y** el vault de Obsidian. (`lisart docs` solo hace el primero; `lisart vault`, solo el segundo.)
3. `lisart validate` — ver que lo que ya está guardado sigue cumpliendo.

La columna aparece sola en la pantalla: los controles se generan desde el esquema, así que la app no duplica el modelo.

Si la columna nueva es **obligatoria** y ya hay filas guardadas, la validación va a marcar todas esas filas. O nace opcional (`required=False`), o se llena el dato antes.

4.2 Agregar una tabla

Igual: se define en `tables.py` y hereda gratis lectura, escritura, validación, documentación y su pestaña en la pantalla.

Lo que hay que decidir al escribirla:

- **capa** (`master` casi siempre; `interim` si se reconstruye desde `raw`);
- `ownership`: `managed` (solo la escribe el código) o `hand_edited` (la mantiene una persona, va con BOM para Excel);
- **llave primaria** y **llaves foráneas**, que son las que evitan el clásico "el pago del alumno 47" cuando el alumno 47 no existe;
- `sort_by`, si el orden del archivo importa para leerlo.

Si la tabla genera identificadores propios (`ALD-0001`), agregar su prefijo en `ID_PREFIXES` de `src/lisart/ids.py`.

4.3 Cambios que se hacen en un solo lugar

CAMBIO	DÓNDE
El texto de un programa cambió en el formulario	`proyeccion.DEL_FORMULARIO`
Los descuentos se aplican en cascada, no sumados	`descuentos.total_de()`
Otra melodía en el navegador de la pantalla	`MELODIA` en `ui/app.py`
Cuántos respaldos se conservan	`COPIAS_A_CONSERVAR` en `backup.py`
Un color de la interfaz	`assets/estilo.css`

4.4 Antes de dar por bueno cualquier cambio

```
make check
```

Corre `ruff` + las pruebas (sin las de pantalla) + `lisart docs --check` + `lisart validate` + `lisart descuentos --revisar` + `lisart guia --check`.

Las pruebas de pantalla llevan marcador `ui` porque tardan ~4 minutos:

```
make test-todo      # todas, incluidas las de pantalla
```

Si tocaste la pantalla, **corre esas también y mira capturas**. Varios defectos de diseño y dos bugs reales solo se vieron mirando, no leyendo.

4.5 Si el cambio afecta a quien usa el sistema

Un cambio que cambia **cómo se usa** algo no está terminado hasta que está escrito. El orden es:

1. Hacer el cambio y probarlo.

2. Actualizar **esta guía** (`docs/guia_de_uso.md`). Si además cambia *qué es* el sistema —una tabla nueva, un proceso nuevo, un pendiente que se cerró—, actualizar también `docs/documentacion_general.md` .
3. `lisart guia` para regenerar los PDF.
4. `make check` , que verifica que quedaron parejos.

5. Cuando algo truena

LO QUE DICE	QUÉ SIGNIFICA	QUÉ HACER
`ERROR <tabla>.<col> fila 42: ... no es un valor valido de tipo 'date'`	Una celda no se puede interpretar. El número es la **línea del archivo**.	Abrir el CSV en esa línea y corregir.
`X fila(s) sin valor en una columna obligatoria`	Falta un dato requerido.	Llenarlo, o el `Column` debería ser `required=False`.
`X valor(es) apuntan a <tabla>.<col> pero no existen alla`	Una referencia rota.	Corregir la clave, o dar de alta lo que falta.
`Valores fuera del catalogo`	Se escribió algo que no está en la lista permitida.	Corregir la escritura, o agregarlo al catálogo si de verdad existe.
`boolean value of NA is ambiguous`	Alguien evaluó una celda vacía de pandas como si fuera verdadero o falso. Es un bug del código, no de los datos.	Usar `ui.app.tiene_valor()` o `pd.isna()`. **Nunca** `if fila["x"]`.
`No se escribio <tabla>, tiene errores`	La escritura se rechazó **antes** de tocar el archivo.	El archivo anterior está intacto. Corregir y repetir.
La pantalla no cierra con `pkill`	El patrón aparece en su propia línea de comando.	`fuser -k 8501/tcp`
Una sección sale a medias y falta lo de abajo	**No se perdió: todavía se está pintando.** La primera carga tarda ~30 s.	Esperar. Mientras trabaja, arriba a la derecha dice "Stop"

LO QUE DICE	QUÉ SIGNIFICA	QUÉ HACER
Una gráfica sale vacía, sin error	En este equipo **ninguna librería de gráficas de Streamlit dibuja** (plotly, altair, ni las nativas).	Dibujar SVG en línea con `st.markdown(..., unsafe_allow_html=True)`.

Regla general: **el sistema prefiere no escribir a escribir algo dudoso.** Si un comando dice que no hizo nada, casi siempre es porque encontró algo que solo una persona puede decidir.

6. Dónde está cada cosa

```
src/lisart/
  tables.py            EL MODELO DE DATOS. Empezar a leer aquí.
  schema.py            las piezas para describir una tabla
  io.py                lectura y escritura tipada y atómica
  validate.py          las revisiones
  ids.py               identificadores y normalización de nombres
  backup.py            respaldos
  guia.py              los documentos de docs/, en PDF
  cli.py               los comandos
  fuentes/             conectores con sistemas de afuera
  procesos/            de materia prima a la verdad de la escuela
  ui/                  las dos pantallas

data/
  raw/                 lo que llegó de afuera, inmutable
  interim/             pasos intermedios, se reconstruyen
  master/              la verdad de la escuela. Esto es lo que se respalda.
  exports/             reportes para gente, desechables
  backups/             copias del estado anterior, fuera de las cuatro capas

docs/
  guia_de_uso.md       esta guía. EL ORIGINAL: aquí se escribe.
  guia_de_uso.pdf      la misma guía en PDF, generada con `lisart guia`
  guia_de_uso.sha256   huella del .md que produjo ese PDF
  documentacion_general.md   qué se construyó y por qué. Para dirección.
  documentacion_general.pdf  su PDF, generado con `lisart guia general`
  mapa_del_codigo.md         jerarquía de los módulos y qué es front y qué back
  mapa_del_codigo.pdf        su PDF, generado con `lisart guia codigo`
  guia_de_entrega.md         cómo pasarle el sistema a otro desarrollador
  guia_de_entrega.pdf        su PDF, generado con `lisart guia entrega`
  convenciones_datos.md  las reglas de la casa, con su porqué
  modelo_de_datos.md   generado por `lisart docs`

vault/                 el sistema como grafo, para abrir en Obsidian
```

Los cuatro documentos que van en PDF

Son cuatro, y cada uno tiene un lector distinto:

DOCUMENTO	CONTESTA	PARA QUIÉN
`guia_de_uso.md`	**cómo se usa** el sistema	quien lo opera cada semana
`documentacion_general.md`	**qué se construyó y por qué**	dirección
`mapa_del_codigo.md`	**por dónde se empieza a leer el código**	quien lo va a mantener
`guia_de_entrega.md`	**cómo se le pasa el sistema a alguien más**	quien entrega y quien recibe, juntos

Los cuatro existen en dos formas: el markdown, que es **el original**, y el PDF, que es la copia que se le pasa a alguien (se abre con doble clic, se imprime, se manda por correo).

El PDF nunca se edita. Se regenera:

```
lisart guia            # regenera los cuatro PDF
lisart guia codigo     # solo uno (guia · general · codigo · entrega)
lisart guia --check    # avisa si algún PDF se quedó atrás de su texto
```

`lisart guia --check` corre dentro de `make check`, así que cambiar un documento y olvidar su PDF **truena la revisión** en vez de pasar inadvertido. Dos documentos que se editan por separado se contradicen en la segunda semana, y el equivocado siempre resulta ser el que alguien tenía impreso.

7. Palabras que se usan aquí

PALABRA	QUÉ QUIERE DECIR
Terminal	La ventana donde se escriben comandos en vez de dar clic. En Windows con WSL se abre buscando "Ubuntu".
Comando	Una línea que se teclea y se ejecuta con Enter. Aquí todos empiezan con `lisart`.
CSV	Un archivo de tabla, en texto plano. Se abre con Excel como cualquier hoja de cálculo. Es donde vive toda la información de la escuela.
Tabla	Una hoja de esas: `personas`, `alumnos`, `clases`... Cada una es un archivo `.csv`.
Registro / fila	Un renglón de una tabla. Una persona, una clase, un descuento.
Capa	En qué etapa está un dato: `raw` (como llegó de afuera), `interim` (a medio procesar), `master` (la verdad de la escuela), `exports` (reportes para gente).
Esquema / modelo	La descripción de qué columnas tiene cada tabla y qué se le permite a cada una. Vive en un solo archivo, `tables.py`.
Validar	Revisar que los datos cumplan esa descripción: que no falten obligatorios, que las fechas sean fechas, que nada apunte a algo que no existe.
Llave	El dato que identifica a algo sin repetirse: `PER-0042` es una persona, `ALD-0001` un renglón de descuento.
Llave foránea	Una columna que apunta a otra tabla. Es lo que evita "el pago del alumno 47" cuando el alumno 47 no existe.
Catálogo	Una lista cerrada de valores permitidos: los espacios, los instrumentos, los descuentos. Se mantiene a mano.
Ingesta	Traer datos de afuera (Jotform, el Excel de la escuela) hacia adentro del sistema.
Respaldo	Una copia del archivo tal como estaba antes de guardar. Se hace sola.
Entorno (`.venv`)	La carpeta donde quedan instalados Python y las piezas del sistema. Se crea una vez, al principio.
Repo / repositorio	La carpeta del proyecto completo, con el código y la documentación.
Pantalla / la app	Las dos ventanas de navegador que abre `lisart ui`.
Regenerar	Volver a producir un archivo desde su fuente. Los generados no se editan a mano: se regeneran.

8. Privacidad

Los datos incluyen nombres, domicilios, teléfonos, firmas e identificaciones de **menores de edad**. Eso manda sobre lo demás.

- La carpeta `data/` **no se sube a ningún lado**, salvo los catálogos, que no llevan datos personales.
- Las contraseñas y llaves de servicios van en el archivo `.env`, que tampoco se sube.
- La pantalla solo escucha en esta computadora: no se puede abrir desde otra.
- **Antes de compartir cualquier archivo de `exports/`, revisar qué columnas lleva.** Un reporte de cobranza puede traer teléfonos y domicilios sin que nadie lo haya pedido.